

Hardware-Constrained Architecture Search for Lane-Change Intention Prediction from Vehicle Time Series

Sepehr Mohammady, [TODO: co-authors, order], [TODO: supervisor]

Abstract—Predicting a driver’s lane-change intention early enough to act on it is a time-series classification problem with hard deployment limits: the model has to run on automotive-grade microcontrollers within tight memory and latency budgets. We apply constrained neural architecture search to the Lane Change Intention Recognition driving dataset [3] (windows of 50 samples over 31 signals) and obtain compact one-dimensional convolutional networks for three tasks: three-class intention classification and time-to-lane-change regression for left and right maneuvers. [TODO: results: accuracy, RMSE, model size, latency on STM32] The searched models improve on the published baseline (Transformer, 0.51 s time-to-lane-change RMSE, deployed in FP32 [1]) while using [TODO: X] $\times$ less memory, and we report measured, not estimated, footprint and latency on the same STM32 targets after int8 quantization.

Index Terms—Neural architecture search, lane-change intention, time-series classification, TinyML, embedded inference, STM32.

I. INTRODUCTION

[TODO: Motivation: intention prediction as advance warning for ADAS; why on-device inference (latency, privacy, cost, no connectivity assumptions); the gap: published DMIR models target accuracy, not deployment budgets.]

Contributions:

- 1) a constrained search space and objective tuned to one-dimensional driving signals under microcontroller budgets;
- 2) models that beat the published DMIR baseline on all three tasks at a fraction of its footprint [TODO: verify once results exist];
- 3) measured deployment numbers (flash, RAM, latency) on [TODO: STM32 part], with the full pipeline released as open source.

II. RELATED WORK

A. Lane-change intention prediction

[TODO: The LC dataset and framework baseline [1] (TTL regression, Transformer RMSE 0.51 s, FP32 deployment on STM32H7B3/F401); earlier HighD-based TTL work (RMSE 0.63–0.7); MicroNAS for time series and TinyTNAS as closest NAS relatives.]

The authors are with the ELIOS Lab, Department of Electrical, Electronic and Telecommunications Engineering and Naval Architecture (DITEN), University of Genoa, Genoa, Italy.

This work was supported by the SYNERGIES project [TODO: grant number and official acknowledgment text].

B. Constrained architecture search for microcontrollers

[TODO: muNAS [2], MCUNet/TinyNAS, MicroNets; what transfers to 1D time series and what does not.]

III. DATA

We use the Lane Change Intention Recognition dataset [3] (50 human drivers in the CARLA simulator, 10 Hz, over 3,400 annotated lane changes; MIT license), prepared as fixed windows of $T=50$ samples (5 s) over $F=31$ channels [TODO: confirm channel order and whether channel 31 is fileTime — drop it if so; official feature count is 30]. Classification uses balanced splits of 94,620 / 16,332 / 19,722 windows (train/val/test) over three classes: no intention, right lane change, left lane change. The regression tasks map a window to the time until the lane change, $t \in [0, 4]$ s discretized at 0.1 s, with 39,313 / 7,742 / 8,097 (LCL) and 33,201 / 5,828 / 7,073 (LCR) windows.

The splits are user-independent: we verified by matching raw per-driver session logs against the prepared windows (correlation over 50-sample windows) that the validation and test drivers follow the protocol of [1] with no window leakage across splits. The test splits contain isolated spikes (up to 5×10^6) on the right/left curvatureDx channel pair — a numerical-derivative artefact absent from training; we clip validation and test features to the per-channel training range and leave training data untouched.

IV. METHOD

A. Search space

The search space is built from one-dimensional convolutional blocks: standard, depthwise and 1×1 convolutions with kernel sizes in $\{3, 5, 7\}$ and channel counts up to 128, optional stride-2 and pooling, blocks connected serially or with residual branches, and a head of one to three fully-connected layers. The operator set is restricted to layers the ST Edge AI compiler maps efficiently. [TODO: state the exact ranges once frozen.]

B. Constraints and search algorithm

We use aging evolution: a population of architectures is sampled, the fittest of a random subset is mutated by a network morphism, its child is trained, and the oldest member is retired. Fitness combines four objectives—validation error, peak RAM, model size and multiply-accumulate count—by the random scalarisation of [2]: each is normalised by a user

TABLE I
TEST-SET COMPARISON. THE INTERNAL REFERENCE MODELS (UNPUBLISHED) WERE EVALUATED ON THE SAME TEST SET; CLASSIFICATION ACCURACY IS OUR OWN EVALUATION, THE TRANSFORMER RMSE VALUES ARE AS REPORTED. SIZE IS THE PARAMETER COUNT.

Model	Params	Acc. (%)	LCR	LCL
Reference CNN (cls)	441 k	91.5	—	—
Reference Transf. (reg, RMSE)	333/49 k	—	0.42	0.44
Our DSCNN baseline	10 k	91.5	0.32 [†]	0.33 [†]
Ours searched (cls / reg MAE)	84 k	92.1	0.29 [†]	0.33 [†]
Ours compact	8 k	91.3	—	—
Ours, no turn-signal	21 k	91.1	—	—

[†]MAE (our search optimizes MAE); reference regression is reported as RMSE. Our LCR RMSE is 0.45 and LCL 0.50.

budget and divided by a random weight resampled every round, so a candidate exceeding any budget is penalised first. Budgets were set from the STM32H7B3I-DK. [TODO: state population/sample sizes, rounds, and search cost.]

C. Discovered architecture

The search independently arrives at three microcontroller-friendly choices: depthwise-separable convolutions, a residual connection, and aggressive early downsampling. The classification model reduces the sequence $50 \rightarrow 25 \rightarrow 13 \rightarrow 7 \rightarrow 4$ before its head:

$50 \times 31 \rightarrow \text{MaxPool} \rightarrow \text{DWConv}(s2) \rightarrow \text{Conv}_{1 \times 1}(77) \rightarrow \text{DWConv} \rightarrow \text{DWConv}(s2) \rightarrow \text{Conv}_{1 \times 1} \rightarrow \text{Add} \rightarrow \text{AvgPool} \rightarrow \text{FC}(223) \rightarrow \text{FC}(23) \rightarrow \text{FC}(3)$

This explains the footprint gap quantitatively. Both our model and the reference CNN spend most of their flash in the fully-connected head, so the gain does not come from avoiding a large head; it comes from what enters it. The reference holds full temporal resolution and flattens a 50×128 map into 6400 features, giving a 64×6400 layer of 1.64 MB ($\sim 95\%$ of the model). Ours pools first and flattens only $4 \times 77 = 308$ features, so it affords a *wider* head (223 units) in a ~ 280 KB layer—a $6\times$ reduction. The same downsampling cuts the convolutional work, yielding the $12.4\times$ fewer MACs and the $9.2\times$ measured speed-up of Table II.

V. EXPERIMENTS

A. Setup

[TODO: training schedule, hardware, seeds/repeats, hyperparameters; all runs logged to the released experiments.jsonl.]

B. Results

Table I reports test-set metrics. All searched-model numbers are our own test-set evaluation of the saved models, from searches run to their full 150-round budget. [TODO: add peak-RAM, MACs, and measured on-device latency columns from ST Edge AI; consider re-running with an all-models save policy for a guaranteed-optimal front (see notes).]

For classification the searched model is both smaller and more accurate than the 441 k reference CNN (92.1% at 84 k

TABLE II
MEASURED ON-DEVICE COMPARISON, STM32H7B3I-DK (CORTX-M7, 280 MHz), ST EDGE AI CORE 4.0.1, FP32.

	Reference CNN	Ours	Ours (tiny)
Parameters	441 k	84 k	8 k
Accuracy (%)	91.7	92.1	91.3
Latency (ms)	33.52	3.63	0.79
Flash (KB)	1729	335	37
RAM (KB)	38.3	9.2	9.2
MACC (M)	1.97	0.16	0.03

parameters; a 8 k model still reaches 91.3%). Regression beats the published single-TTLC SOTA (RMSE 0.51) at $2\text{--}3\times$ fewer parameters (our RMSE 0.45 LCR, 0.47 LCL). We do not claim to beat the internal-reference Transformers on regression RMSE (0.42/0.44), which remain competitive small models; an RMSE-aware search narrowed but did not close that gap. Removing the turn-signal channels drops classification accuracy by about one point (92.1% to 91.1%)—evidence that the model anticipates from vehicle dynamics and surrounding traffic rather than reading a declared signal. [TODO: RMSE columns and the internal-reference RMSE (0.42/0.44) are not yet matched because the search optimizes MAE; decide RMSE-aware objective vs MAE-primary framing.]

C. Deployment on STM32

Target platform: STM32H7B3I-DK discovery kit (Cortex-M7 at 280 MHz, 1.4 MB SRAM, 2 MB flash) — the same MCU family used by the baseline framework [1], which deployed FP32 models only. We therefore compare in FP32, like for like, benchmarking both models on the same board through the ST Edge AI Developer Cloud (Core 4.0.1, balanced optimization). Table II reports measured, not estimated, figures.

The searched model is $9.2\times$ faster and $5.2\times$ smaller than the reference CNN while being more accurate; a second operating point on the same front gives up 0.4 points of accuracy for a $42\times$ speed-up and a $47\times$ smaller model that infers in under a millisecond.

The per-layer breakdown explains why. Both our model and the reference spend most of their flash in the fully-connected head, so the gain is not from avoiding a large head but from what enters it: the reference flattens a 50×128 map into 6400 features, giving a 64×6400 layer of 1.64 MB ($\sim 95\%$ of the model), whereas the searched models pool first and flatten only 308 (or 98) features.

RAM behaves differently and is worth stating plainly: both searched models measure ~ 9.2 KB regardless of a $9\times$ parameter gap, because the 50×31 float32 input tensor alone occupies 6.2 KB. On this task RAM is input-bound rather than model-bound, which is why the RAM advantage ($\sim 4\times$) is far smaller than the flash advantage, and why an int8 input—not a smaller network—is the lever that would reduce it.

On quantization, the ST toolchain accepts only float32/int8/uint8 tensors. Full int8 shrinks the classifier to 110 KB but costs accuracy on these wide-dynamic-range inputs (92.1% $\rightarrow$ 86.9%; LCR MAE 0.287 $\rightarrow$ 0.449). An int16 $\times$ 8 scheme (int8 weights, int16 activations) preserves

accuracy almost exactly offline ($92.1\% \rightarrow 92.2\%$; LCR MAE $0.287 \rightarrow 0.286$), but the ST toolchain does not support it—its documentation restricts weight and activation tensors to float32, int8 and uint8—so we report it as an offline bound rather than a deployed configuration. [TODO: cite ST Edge AI Core 4.0 documentation (quantization / supported ops, retrieved 2026-07-14); confirm against the pending int16x8 benchmark.] [TODO: insert ST Edge AI measured latency / flash / peak SRAM per board (STM32H7B3I-DK and NUCLEO-F401RE); see docs/research/deployment.md.]

VI. CONCLUSION

[TODO: two or three sentences; no new claims.]

REFERENCES

- [1] L. Forneris *et al.*, “A deployment-oriented simulation framework for deep learning-based lane change prediction,” *IEEE Signal Process. Lett.*, vol. 33, pp. 136–140, 2026, doi: 10.1109/LSP.2025.3638676.
- [2] E. Liberis, Ł. Dudziak, and N. D. Lane, “ μ NAS: Constrained neural architecture search for microcontrollers,” in *Proc. 1st Workshop Mach. Learn. Syst. (EuroMLSys)*, 2021, pp. 70–79.
- [3] L. Forneris *et al.*, “Lane change intention recognition dataset,” Zenodo, 2025, doi: 10.5281/zenodo.16686054.
- [4] L. Forneris *et al.*, “Setting up a lightweight simulation environment for automated driving dataset collection,” in *Applications in Electronics Pervading Industry, Environment and Society (ApplePies)*, Springer LNEE, 2024, pp. 156–163, doi: 10.1007/978-3-031-84100-2_19.